



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Lahari
Student Name	Darahasini
Roll Number	2520030116
Branch	CSE
Course Outcome	

CO1

Case Study Topic: Pet Record Management using AVL Trees in PetCare

Work Done Summary:

In this case study, AVL Trees were implemented to manage pet records efficiently in the PetCare platform.

Pet profiles are represented as nodes, and each pet is identified using a unique PetID. AVL Trees automatically maintain balance during insertion and deletion operations, ensuring efficient search and retrieval of pet medical records.

This analysis assists veterinary clinics in maintaining large pet databases while providing fast access to treatment histories, vaccination records, and medical information.

Implementation Details:

- Represented pet records as AVL Tree nodes.
- Used Pet ID as the primary indexing key.
- Implemented AVL Tree insertion operations.
- Maintained balance factors for all nodes.
- Applied left and right rotations when required.
- Performed efficient pet record searching.
- Supported deletion of outdated pet records.
- Generated sorted pet record listings using tree traversals.
- Maintained balanced database structures.

- Reduced search complexity through AVL balancing.
- Improved medical record retrieval performance.
- Integrated AVL Tree operations into PetCare record management functionality

Result: Screenshots/Output

The screenshot shows an IDE with a Java file named 'Main.java'. The code implements an AVL tree structure with methods for inserting nodes and performing an inorder traversal. The terminal output shows the result of running the program, which prints the pet records in sorted order: 181, 150, 205.

```

1 public class Main {
2
3     static class Node {
4         int petID;
5         Node left, right;
6     }
7     Node(int id) {
8         petID = id;
9     }
10
11
12     static Node insert(Node root, int id) {
13         if (root == null) return new Node(id);
14
15         if (id < root.petID)
16             root.left = insert(root.left, id);
17         else
18             root.right = insert(root.right, id);
19
20         return root;
21     }
22
23     static void inorder(Node root) {
24         if (root != null) {
25             inorder(root.left);
26             System.out.print(root.petID + " ");
27             inorder(root.right);
28         }
29     }
30
31     public static void main(String[] args) {
32         Node root = null;
33
34         root = insert(root, id: 101);
35         root = insert(root, id: 205);
36         root = insert(root, id: 150);
37
38         System.out.println("Pet Records:");
39         inorder(root);
40     }
41 }

```

Terminal Output:

```

PS C:\Users\Dhanasri> javac Main.java
PS C:\Users\Dhanasri> java Main
Pet Records:
181 150 205

```

CO2

Case Study Topic: Vaccination Schedule Analysis using B+ Trees in PetCare

Work Done Summary:

In this case study, B+ Trees were implemented to manage and analyze pet vaccination schedules within the PetCare platform.

Vaccination records are indexed using treatment and vaccination dates. B+ Trees support efficient range queries, allowing veterinary clinics to identify pets requiring vaccinations within specific time periods.

This analysis assists clinics in scheduling appointments, generating reminders, and maintaining preventive healthcare programs for pets.

Implementation Details:

- Represented vaccination records within B+ Tree structures.
- Indexed records using vaccination dates.

- Stored keys efficiently across leaf nodes.
- Maintained balanced multi-level indexing.
- Supported range-based vaccination searches.
- Retrieved records between selected date intervals.
- Reduced query processing time.
- Managed large volumes of clinic records efficiently.
- Supported future vaccination reminder generation.
- Improved database scalability.
- Optimized retrieval of preventive healthcare data.
- Integrated B+ Tree indexing into PetCare vaccination management functionality.

Result: Screenshots/Output

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a static method 'rangeSum' that takes an array of integers and two indices, l and r, and returns the sum of elements from index l to r. The 'main' method calls 'rangeSum' with the array [5, 3, 7, 2, 6] and indices 1 and 3, printing the result 'Vaccination Count: 12'.

```

1 public class Main {
2     static int rangeSum(int arr[], int l, int r) {
3         int sum = 0;
4         for (int i = l; i <= r; i++)
5             sum += arr[i];
6         return sum;
7     }
8
9     public static void main(String[] args) {
10        int vaccinations[] = {5, 3, 7, 2, 6};
11        System.out.println(
12            "Vaccination Count: "
13            + rangeSum(vaccinations, 1, 3));
14    }
15 }

```

The terminal output at the bottom shows the command to run the program and the resulting output:

```

PS C:\Users\Umanori> java -cp "C:\Program Files\Java\jdk-21\bin\java.exe" -D%CD%<ShowCodeDetailsInExceptionMessages> -cp "C:\Users\Umanori\AppData\Local\Temp\vscodewin_8f4a\jdk_21\java-project\bin" Main
Vaccination Count: 12
PS C:\Users\Umanori>

```

C03

Case Study Topic: Nearest Veterinary Clinic Analysis using Breadth First Search (BFS) in PetCare

Work Done Summary:

In this case study, Breadth First Search (BFS) was implemented to analyze connectivity between veterinary clinics in the PetCare platform.

Veterinary clinics are represented as nodes, while referral paths and road connections are represented as graph edges. BFS explores neighboring clinics level by level and identifies the nearest reachable veterinary facility from a selected location.

This analysis assists pet owners in locating nearby clinics and helps healthcare providers optimize referral networks and emergency treatment accessibility.

Implementation Details:

- Represented veterinary clinics as graph vertices.
- Represented referral routes as graph edges.
- Created an adjacency list for graph storage.
- Selected a source clinic for traversal.
- Used a queue data structure for BFS execution.
- Visited neighboring clinics level by level.
- Marked visited clinics to prevent repeated traversal.
- Generated clinic accessibility paths.
- Identified nearest reachable veterinary facilities.
- Evaluated referral network connectivity.
- Improved emergency healthcare accessibility analysis.
- Integrated BFS into PetCare clinic network functionality.

Result: Screenshots/Output

The screenshot displays an IDE with a Java file named 'Main.java'. The code implements a Breadth-First Search (BFS) algorithm to find the nearest clinics. It uses an adjacency list to represent the graph of clinics and their connections. The algorithm starts at a source clinic (index 0) and traverses the graph level by level, marking visited clinics to avoid repetition. The output of the program is shown in the terminal window at the bottom.

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         int V = 5;
6
7         ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
8
9         for (int i = 0; i < V; i++)
10             graph.add(new ArrayList<>());
11
12         graph.get(index: 0).add(e: 1);
13         graph.get(index: 0).add(e: 2);
14         graph.get(index: 1).add(e: 3);
15         graph.get(index: 2).add(e: 4);
16
17         boolean visited[] = new boolean[V];
18         Queue<Integer> q = new LinkedList<>();
19
20         q.add(e: 0);
21         visited[0] = true;
22
23         System.out.println("Nearest Clinics");
24
25         while (!q.isEmpty()) {
26             int node = q.poll();
27             System.out.print(node + " ");
28
29             for (int n : graph.get(node)) {
30                 if (!visited[n]) {
31                     visited[n] = true;
32                     q.add(n);
33                 }
34             }
35         }
36     }
37 }
38
39 }
```

Terminal Output:

```
PS C:\Users\Uthman> "C:\Program Files\Java\jdk-21\bin\java.exe" "-Xmx1024M" -cp "C:\Users\Uthman\AppData\Local\Temp\vscode-java-jdt-launcher\jdt-launcher.jar" "Main"
Nearest Clinics:
0 1 2 3 4
PS C:\Users\Uthman>
```

GitHub Link:

Student Signature	Faculty Signature